Omnia Partners - Build Document

Omnia Partners — Build Document

WhereScape RED → Coalesce Migration

Client Omnia Partners
Source System WhereScape RED on Snowflake (OMNIAPARTNERSDW)
Target System Coalesce on Snowflake
Date 2026-06-27
Status POC Complete — Ready for Import

---

1. Current State

Omnia Partners operates a WhereScape RED data warehouse on Snowflake (database: OMNIAPARTNERSDW) covering sales, revenue, contracts, suppliers, customers, UC spend, and OPUS e-commerce. The warehouse consists of:

Object Type	Count	Columns
Load Tables (Sources)	90	2,360
Stage Tables	117	5,444

Object Type	Count	Columns
Dimensions	74	1,980
Fact Tables	22	490
Views	125	5,052
Aggregates	37	986
Data Stores	13	640
Total	**478**	**16,952**

Key characteristics: - 174 multi-source objects (JOINs to multiple tables) - 474 objects with explicit JOIN conditions - 12 storage locations across 3 Snowflake databases - 17 scheduler jobs - 291 scripts (272 UPDATE_, 17 SCRIPT_, 1 CUSTOM_, 1 POST_) - Only 3 scripts show evidence of manual modification

---

2. Build Approach

2.1 Automated Conversion

The migration uses our proven automated tooling (`parse_omnia.py`) which:

1. **Parses** the WhereScape metadata CSV (16,952 column-level lineage rows)
2. **Resolves** object and column names to original WhereScape case
3. **Generates** Coalesce-native YAML nodes with full:
    - Column definitions and data types
    - Source column references (UUID-linked lineage)
    - JOIN conditions using `{{ ref() }}` syntax
    - Column transforms (SUM, COUNT, COALESCE, CAST, etc.)
    - Business key identification
    - System audit columns (DSSCreateTime, DSSUpdateTime)

4. **Detects and resolves** circular dependencies via Tarjan's SCC algorithm
5. **Validates** with `coa validate` — target: 0 errors

2.2 Build Status (POC Complete)

Check	Status
Nodes generated	478 ✓
`coa validate` errors	0 ✓
Circular dependencies	0 (resolved via ref_no_link) ✓
Duplicate column IDs	0 ✓
Column source mappings	Valid ✓
Column dependencies	Valid ✓
Column references	Valid ✓
Data type warnings	285 (width mismatches — acceptable)

---

3. Node Type Configuration

Node Type	ID	Materialization	Key Features
Stage	Stage	table	truncateBefore, insertStrategy=INSERT
Dimension	Dimension	table	businessKeyColumns, SCD system columns
Fact	Fact	table	System date columns
View	View	view	insertStrategy=UNION

Node Type	ID	Materialization	Key Features
Aggregate	390	table	GROUP BY ALL in joinCondition
Persistent Stage	persistentStage	table	businessKeyColumns, MERGE logic
Dimension View	DimensionView	view	Role-playing dimensions

---

4. Storage Locations

Location	Database	Schema	Usage
LOAD	OMNIAPARTNERSDW	LOAD	Landing/ source tables
STAGE	OMNIAPARTNERSDW	STAGE	Integration staging
DIM	OMNIAPARTNERSDW	DIM	Dimensions
FACT	OMNIAPARTNERSDW	FACT	Fact tables
ODS	OMNIAPARTNERSDW	ODS	Data stores
OPC	OMNIAPARTNERSDW	OPC	OMNIA Partners Connect
REPORTS	OMNIAPARTNERSDW	REPORTS	Reporting views & aggregates

Location	Database	Schema	Usage
OP	OMNIAPARTNERSDW	OP	Operations
DATARAILS	OMNIAPARTNERSDW	DataRails	DataRails integration
TABLEAU	OMNIAPARTNERSDW	TABLEAU	Tableau views
OMNIACONNECT_OPC	OMNIACONNECT	OPC	Connect platform
DATALAKE_FORCE	OPDataLake	FORCE	Force data lake
DATALAKE_VIZIENT	OPDataLake	Vizient	Vizient data lake

---

5. Design Patterns Applied

5.1 Dimension Key Lookups (Stage _AddKeys tables)

Stage tables that look up surrogate keys from dimensions use: - `LEFT JOIN` to each dimension via `ref_no_link` (breaks circular dependency) - `COALESCE("<Dim>"."<Key>", 0)` transform for unmatched rows (NULL → 0) - Example: `StageAccount_AddKeys` LEFT JOINs to AccountGeography, Organization, Sector, etc.

5.2 Fact Tables

Fact tables source ALL columns (including dimension keys) from their prior staging table: - FROM clause references the `*_AddKeys` stage only - Key columns mapped to the staging table (not dimensions) - No dimension JOINs on fact tables - Example: `SalesAndRevenue` reads from `StageSalesMerge_AddKeys`

5.3 Views with WHERE Clauses

Views may have: - Full FROM/JOIN/ON clauses (parsed from `obj_join`) - WHERE-only clauses (FROM derived from primary column source) - Example: `AccountHierarchy_OMNIA` = `FROM AccountHierarchy WHERE ChannelPartnerFeeSharePartnerId = 1`

5.4 Aggregates

Aggregate tables use: - Node type `"390"` (custom copy of Fact) - `GROUP BY ALL` appended to joinCondition - Transforms: `SUM()`, `COUNT()`, `MAX()`, `AVG()`, `DATEDIFF()` - No system columns added

5.5 Circular Dependency Resolution

Cycles broken using `ref_no_link` at the following points: - Stage tables → Account dimension (28 nodes) - Stage tables → AccountHierarchy_OMNIA (3 nodes) - Views → Account, Revenue, Sales, Feeshare (various) - DIM views → Account (AccountHierarchy_OMNIA) - Stage tables → Contact dimension (2 nodes)

---

6. Column Mapping Rules

Condition	Mapping
Column has a transform expression	`columnReferences: []`, `transform: <expression>`
Column maps 1:1 to source column	`columnReferences: [{stepCounter, columnCounter}]`, `transform: ""`
Column has neither source nor transform	`columnReferences: []`, `transform: "NULL"`

Condition	Mapping
DSSCreateTime / DSSUpdateTime (unmapped)	`transform: "CAST(CURRENT_TIMESTAMP AS TIMESTAMP)"`
Dimension key lookup (Stage AddKeys)	`transform: 'COALESCE("<Dim>"."<Key>", 0)'`
Fact key column	Redirected to staging table (not dimension)
Transform with trailing `AS ALIAS`	Stripped (Coalesce adds its own AS)

---

7. Scripts & Custom Code

Category	Count	Disposition
Generated UPDATE_ scripts	272	Absorbed into node generation (metadata = source of truth)
SCRIPT_ load scripts	17	Infrastructure — map to Coalesce job configuration
CUSTOM_SalesAndRevenue	1	Empty placeholder (0 code) — no action
Modified UPDATE_DsSalesforceContracts	1	Persistent Stage handles MERGE natively
Modified POST_LoadReportingPeriods	1	Maps to postSQL config on target node

8. Validation Results

```
coa validate output:
  ✔ Schema Validation (488 files)
  ✔ Storage Locations
  ✔ Storage Mappings
  ✔ Node Location Data
  ✔ Node Type Validity
  ✔ Node Type Availability
  ✔ Node Type Default Locations
  ✔ Duplicate Nodes
  ✔ Duplicate Subgraphs
  ✔ Duplicate Jobs
  ✔ Column Sources
  ✔ Column Source Mappings
  ✔ Column Dependencies
  ✔ Column References
 ⚠ Column Data Types (285 warnings)
  ✔ Column Names
  488 files  0 errors  285 warnings
```

The 285 warnings are data type width mismatches (e.g., `VARCHAR` vs `VARCHAR(18)`, `NUMBER` vs `NUMBER(38,0)`) — acceptable and non-blocking.

---

9. Deployment Steps

1. **Import nodes** — `coa` CLI push or git commit to Coalesce workspace
2. **Verify graph** — confirm all nodes render in the Coalesce DAG view
3. **Deploy to DEV** — `coa deploy` to development environment
4. **Run pilot** — execute Sales & Revenue subject area pipeline

5. **Reconcile** — compare row counts and checksums to legacy WhereScape output
6. **Wave deployment** — deploy remaining subject areas in dependency order
7. **Configure jobs** — rebuild 17 scheduler jobs as Coalesce jobs
8. **Parallel run** — both systems active, daily reconciliation
9. **Cutover** — disable WhereScape, Coalesce primary

---

10. Files Delivered

```
migration_poc/
├── data.yml                    # Default storage mapping
├── locations.yml               # 13 storage locations
├── workspace.yml               # Location → database/
schema mapping
├── nodeTypes/                  # 7 node type definitions
│   ├── Aggregate-390/
│   ├── Dimension-Dimension/
│   ├── DimensionView-DimensionView/
│   ├── Fact-Fact/
│   ├── PersistentStage-persistentStage/
│   ├── Stage-Stage/
│   └── View-View/
├── nodes/                      # 478 generated node YAML
files
│   ├── LOAD-*.yml              (90 source nodes)
│   ├── STAGE-*.yml             (117 stage nodes)
│   ├── DIM-*.yml               (74 dimensions)
│   ├── FACT-*.yml              (22 facts)
│   ├── REPORTS-*.yml           (aggregates + reporting
views)
│   ├── ODS-*.yml               (13 data stores)
│   └── ...                     (other locations)
├── scripts/
```

```
│   ├── parse_omnia.py          # Main parser (re-runnable)
│   └── extract_csv_from_wst.py # WST → CSV extractor
├── PROPOSAL.md / .pdf          # Client proposal
├── PROJECT_PLAN_TEMPLATE.md    # Project plan template
└── WHERESCAPE_MIGRATION_GUIDE.md  # Technical reference
```

---

11. Known Limitations

1. **60 orphan views** — no JOIN/source info in CSV (role-playing dimensions, external views). Require manual wiring.
2. **Data type warnings** — 285 width mismatches. Non-blocking; can be refined per-column if needed.
3. **Column-level lineage gaps** — columns with `transform: "NULL"` have no lineage. These are system columns or computed columns with no source in the metadata.
4. **Scheduler jobs** — 17 jobs need manual recreation as Coalesce jobs (schedules + dependencies not in the deployment export).

---

12. Next Steps

1. Import 478 nodes into Coalesce workspace
2. Verify graph rendering and resolve any UI-reported issues
3. Deploy pilot subject area (Sales & Revenue)
4. Begin reconciliation against legacy WhereScape output